

BookRAG

End-to-End Agentic RAG Chatbot

Complete Build Specification for Antigravity

Primary goal: Achieve high-quality, grounded RAG responses with zero hallucination. Latency is not a concern at this stage — correctness is everything.

Antigravity instruction: Read this entire document before writing a single line of code. Make all reasonable assumptions and proceed without asking the user anything. Where genuine ambiguity exists, pick the more robust/defensive option and leave an `# ASSUMPTION:` comment in the code. The goal is a fully runnable project after one generation pass.

0. Project Overview

A Streamlit chatbot that answers questions about 11 personal-finance/wealth books using a LangGraph-powered agentic RAG pipeline. Everything runs locally. The pipeline has two paths:

- **General path** — conversational queries (greetings, meta-questions) answered directly by the lightweight LLM without touching the vector DB.
- **RAG path** — specific book-related queries go through a rigorous multi-stage pipeline: retrieval (Chroma dense + BM25 sparse) → RRF fusion → generation → hallucination evaluation → one mandatory retry with query expansion → final response.

NOTE: Because quality is the top priority, the RAG loop always attempts a retry on first failure. There is no early exit. The pipeline runs up to 2 full generate-then-evaluate cycles before producing a final answer.

1. Directory Structure

Create this layout before writing any code:

```
bookrag/
████ README.md
████ requirements.txt
████ .env.example
████ ingest.py           # One-time PDF ingestion → Chroma + BM25 index
████ pipeline.py         # LangGraph RAG pipeline
████ app.py              # Streamlit UI
████ utils/
■   █████ __init__.py
■   █████ embeddings.py  # BGE embedding wrapper
■   █████ retrieval.py   # Chroma + BM25 retrieval + RRF fusion
■   █████ history.py     # Chat history + rolling summarisation
████ chroma_db/          # Auto-created at ingest time (gitignore)
████ books/              # User places 11 PDFs here (gitignore)
```

2. Environment & Dependencies

.env.example

```
# Copy to .env and fill in
OLLAMA_BASE_URL=https://e7a8-154-192-139-73.ngrok-free.app
# This ngrok URL rotates every session on the free tier.
# Update this variable before every run.
```

requirements.txt

```
streamlit>=1.35.0
langgraph>=0.2.0
langchain>=0.2.0
langchain-community>=0.2.0
langchain-openai>=0.1.0
chromadb>=0.5.0
sentence-transformers>=3.0.0
rank-bm25>=0.2.2
pypdf>=4.0.0
```

```
python-dotenv>=1.0.0
tiktoken>=0.7.0
```

3. Models & Endpoints

Role	Model	Notes
Rewrite query + route decision	qwen3.5:0.8b	Lightweight; also runs summarisation
General query answering	qwen3.5:2b	Mid-tier; handles conversational replies
RAG generation + grounding eval	qwen3.5:4b	Heavy lifter; two separate calls per RAG turn
Embeddings	BAAI/bge-small-en-v1.5	Local CPU via sentence-transformers; no GPU needed

ASSUMPTION: All three Ollama models are already pulled and served at the ngrok URL. The endpoint is OpenAI-compatible. Use `langchain_openai.ChatOpenAI` with `openai_api_key='ollama'` (dummy key) and `base_url=f'{OLLAMA_BASE_URL}/v1'`. Model names passed as exact strings: 'qwen3.5:0.8b', 'qwen3.5:2b', 'qwen3.5:4b'.

4. Ingestion Script — ingest.py

One-time offline script. The user runs *python ingest.py* once before launching the app. This is the foundation of RAG quality — every decision here directly affects retrieval precision.

Step 1 — Discover PDFs

Scan `./books/` for all `*.pdf` files. Print the list found. Exit with a clear error if 0 PDFs found.

Step 2 — Extract Text

Use `pypdf.PdfReader`. For each page call `page.extract_text()`. Skip pages that return empty or None. Strip excessive whitespace but preserve paragraph breaks (double newlines). Tag each book's extracted text with its filename.

Step 3 — Chunking (critical for RAG quality)

Use `langchain.text_splitter.RecursiveCharacterTextSplitter`. Settings:

- **chunk_size=800** — larger chunks preserve full arguments/concepts from dense nonfiction
- **chunk_overlap=150** — generous overlap ensures no idea is split across chunk boundaries
- **separators=["\n\n", "\n", ". ", " ", "", ""]** — respects paragraph structure first

Tag each chunk's metadata: `{"source": "filename.pdf", "chunk_id": int, "book_title": str}`. Derive a human-readable `book_title` by stripping the `.pdf` extension and cleaning underscores/hyphens.

Step 4 — Embed (BGE asymmetric)

Use `BGEEmbeddings` from `utils/embeddings.py`. Embed all chunks as passages (no prefix). Batch size: 32. Print progress every 100 chunks.

Step 5 — Chroma Vector Store

Persist to `./chroma_db/` using `chromadb.PersistentClient`. Collection name: `'bookrag'`. Distance metric: `cosine` (set `metadata={"hnsw:space": "cosine"}`). If the collection already exists, delete and recreate it for idempotent re-ingestion.

Step 6 — BM25 Sparse Index

After Chroma, build a BM25 index using `rank_bm25.BM25Okapi`. Tokenise chunks by whitespace (simple but effective). Save three pickle files:

- **`chroma_db/bm25.pkl`** — the `BM25Okapi` object
- **`chroma_db/bm25_corpus.pkl`** — list of raw chunk text strings, same order as BM25
- **`chroma_db/corpus_metadata.pkl`** — list of `{"source", "book_title", "chunk_id"}` dicts in the same order. Critical: BM25 only returns indices; this maps them back to book names.

Print summary: number of books processed, total chunks, time taken.

5. Embeddings Utility — `utils/embeddings.py`

BGE models are asymmetric — queries must be prefixed, passages must not. This is the most commonly missed detail with BGE and the #1 cause of poor retrieval quality.

```
from sentence_transformers import SentenceTransformer

QUERY_PREFIX = "Represent this sentence for searching relevant passages: "

class BGEEEmbeddings:
    """
    Wrapper around BAAI/bge-small-en-v1.5.
    Implements chromadb EmbeddingFunction interface.
    """
    def __init__(self):
        # ASSUMPTION: model downloads to ~/.cache/huggingface/ on first run.
        self.model = SentenceTransformer("BAAI/bge-small-en-v1.5")

    def embed_passages(self, texts: list[str]) -> list[list[float]]:
        """For indexing book chunks — NO prefix."""
        return self.model.encode(
            texts, normalize_embeddings=True, batch_size=32
        ).tolist()

    def embed_query(self, text: str) -> list[float]:
        """For retrieval at query time — WITH prefix."""
        return self.model.encode(
            QUERY_PREFIX + text, normalize_embeddings=True
        ).tolist()

    def __call__(self, input: list[str]) -> list[list[float]]:
        """chromadb EmbeddingFunction interface (passages)."""
        return self.embed_passages(input)
```

6. Retrieval Utility — `utils/retrieval.py`

The RetrieverClient loads Chroma and BM25 once and exposes a single retrieve_and_fuse() function. Because quality is the priority, retrieve top-15 candidates from each index before fusion, then return top-8 fused results (more context = less hallucination).

Dense Retrieval (Chroma)

Query Chroma with the BGE-embedded query (with prefix). Return top-15 results with their distances. Convert to similarity scores: $\text{score} = 1 - \text{distance}$ (cosine distance after normalisation is in $[0,1]$).

Sparse Retrieval (BM25)

Load bm25.pkl. Tokenise the (rewritten) query by whitespace. Get BM25 scores for all chunks. Return top-15 indices and scores.

RRF Fusion Formula

```
rrf_score(doc, k=60) = sum(1 / (k + rank_i) for each list the doc appears in)
# rank is 1-indexed. k=60 is the standard constant.
```

Merge the two top-15 lists. Compute RRF scores. Return top-8 fused documents (text + source + book_title + rrf_score), sorted descending by rrf_score.

Function Signature

```
def retrieve_and_fuse(
    query: str,
    n_dense: int = 15,
    n_sparse: int = 15,
    n_final: int = 8
) -> list[dict]:
    """
    Returns list of dicts with keys:
    - 'text': str
    - 'source': str (filename)
    - 'book_title': str
    - 'rrf_score': float
    Sorted descending by rrf_score.
    """

class RetrieverClient:
    def __init__(self):
        # Load Chroma, BM25, corpus metadata – called once via @st.cache_resource
        ...

    def retrieve(self, query: str, extra_docs: list[dict] = None) -> list[dict]:
        """
        On retry, pass extra_docs (from first attempt).
        New results are merged with extra_docs before RRF, then top-8 returned.
        This ensures the retry always has MORE context, never less.
        """
        ...
```

NOTE: On retry, merge first-attempt docs with new retrieval results before re-running RRF. Deduplicate by text hash. This cumulative context strategy is the key to reducing hallucination on the second attempt.

7. Chat History — utils/history.py

Two separate concerns: what the UI shows (full history always) vs. what the LLM receives (bounded context via rolling summarisation).

```
class ChatHistory:
    def __init__(self, llm_client):
        self.turns: list[dict] = []          # full display history
        self.summary: str = ""               # rolling compressed summary
        self.unsummarised_start: int = 0     # index where unsummarised turns begin
        self.llm = llm_client                # qwen3.5:0.8b client

    def add_turn(self, role: str, content: str):
        self.turns.append({"role": role, "content": content})
        # Summarise every 10 turns
        if len(self.turns) % 10 == 0:
            self._summarise()

    def _summarise(self):
        batch = self.turns[self.unsummarised_start : self.unsummarised_start + 10]
        text = "\n".join(f"{t['role'].upper(): {t['content']}" for t in batch)
        prompt = (
            "Summarise this conversation in 2-4 sentences, "
            "capturing key topics and conclusions:\n\n" + text
        )
        response = self.llm.invoke([{"role": "user", "content": prompt}])
        new_summary = response.content.strip()
        self.summary = (
            f"{self.summary} Later: {new_summary}" if self.summary else new_summary
        )
        self.unsummarised_start += 10

    def get_context_messages(self) -> list[dict]:
        """Context sent to any LLM call. Always bounded."""
        messages = []
        if self.summary:
            messages.append({
                "role": "system",
                "content": f"Summary of earlier conversation: {self.summary}"
            })
        messages.extend(self.turns[self.unsummarised_start:])
        return messages
```

8. LangGraph Pipeline — pipeline.py

The pipeline is a LangGraph StateGraph. Build it inside a `build_graph()` function that is cached by Streamlit. All LLM clients are created inside this function.

8.1 State Schema

```
from typing import TypedDict, Optional

class RAGState(TypedDict):
    raw_query: str
    rewritten_query: str
    route: str                # "general" | "rag"
```

```

retrieved_docs: list[dict]
first_attempt_docs: list[dict]    # saved from attempt 1 for retry merging
generated_answer: str
is_grounded: bool
grounding_reason: str             # one-sentence reason from evaluator
retry_count: int                  # 0 or 1
expanded_query: str
final_answer: str
disclaimer: str
chat_context: list[dict]          # from ChatHistory.get_context_messages()

```

8.2 Node: rewrite_and_route

Model: qwen3.5:0.8b

Prompt the model to output ONLY a JSON object with two fields. Strip markdown code fences before parsing. On parse failure, default to route='rag' and rewritten_query=raw_query.

System:

You are a query processor. Respond ONLY with valid JSON, no explanation, no markdown fences.

User:

Given the user query below, do two things:

1. Rewrite it to be clearer and more search-friendly for a personal finance book collection. Keep the meaning identical. If the query is already clear, leave it mostly unchanged.
2. Classify the route:
 - "general" if the query is conversational (hi, hello, thanks, how are you, what can you do, who made you, etc.)
 - "rag" if the query is about finance, wealth, investing, money, books, or any substantive topic that might be answered from a book.

Respond ONLY with:

```
{"rewritten_query": "...", "route": "general or rag"}
```

User query: {raw_query}

8.3 Node: retrieve

No LLM. Pure Python retrieval node.

- On first attempt (retry_count==0): call retriever.retrieve(rewritten_query). Store results in retrieved_docs AND first_attempt_docs.
- On retry (retry_count==1): call retriever.retrieve(expanded_query, extra_docs=first_attempt_docs). This merges old and new docs before RRF — cumulative context is the key anti-hallucination strategy on retry.

8.4 Node: generate

Model: qwen3.5:4b

System:

You are a knowledgeable assistant specialising in personal finance and wealth-building. Answer the user's question using ONLY the provided book excerpts below. If the excerpts do not contain enough information to answer confidently, say so honestly rather than guessing. Be thorough and cite the source book name when referencing specific advice or data points.

[chat_context messages included before the user message for conversational continuity]

User:

Question: {rewritten_query}

Book excerpts:

[For each doc: "Source: {book_title}\n{text}\n--"]

Answer based strictly on the excerpts above:

8.5 Node: evaluate_grounding (CRITICAL — anti-hallucination gate)

Model: qwen3.5:4b (separate call from generation)

This is the most important node. The evaluator is instructed to be strict — any claim not directly supported by the excerpts is flagged as ungrounded.

System:

You are a strict fact-checker. Your job is to verify that every factual claim in the answer is directly supported by the provided source excerpts.

Respond ONLY with valid JSON, no explanation, no markdown fences.

User:

Answer to evaluate:

{generated_answer}

Source excerpts used to generate this answer:

[same formatted excerpts as passed to generate node]

Instructions:

- Check each factual claim in the answer against the excerpts.
- If ANY claim cannot be traced to a specific excerpt, mark is_grounded as false.
- Conversational filler ("Based on the books...", "According to...") does not count as a claim — only factual assertions matter.
- Be strict. It is better to flag and retry than to pass a hallucinated answer.

Respond ONLY with:

{"is_grounded": true or false, "reason": "one sentence explaining your decision"}

Parse response. Strip code fences first. On parse failure, set is_grounded=False (conservative — prefer retry over passing a potentially hallucinated answer).

8.6 Node: expand_query

Model: qwen3.5:0.8b

Only reached when retry_count==0 and is_grounded==False.

System:

You are a search query expander. Respond ONLY with the expanded query string, nothing else.

User:

The original query: {raw_query}

The rewritten query: {rewritten_query}

Grounding failure reason: {grounding_reason}

The RAG system failed to find fully grounded information. Generate an expanded version of the rewritten query by:

- Adding 4-6 relevant synonyms or related financial concepts
- Including alternative phrasings of the core question
- Keeping it as a single coherent query string

Respond with ONLY the expanded query, no explanation.

Store result in expanded_query. Set retry_count=1.

8.7 Node: general_answer

Model: qwen3.5:2b

System:

You are a helpful, friendly assistant. Answer conversationally and concisely.

You can also answer general questions about personal finance if the user seems curious.

[chat_context messages for continuity]

User: {raw_query}

8.8 Node: build_final_answer

Pure Python. No LLM. Assembles final_answer and disclaimer:

- **route=='general':** final_answer = generated_answer, disclaimer = "
- **route=='rag' and is_grounded:** final_answer = generated_answer, disclaimer = "
- **route=='rag' and not is_grounded (after retry):** append disclaimer below

```
disclaimer = (
    "\u26a0\ufe0f Note: I could not fully verify this answer against the source books "
    "after two attempts. Some claims may not be directly grounded in the texts. "
    "Please treat this response with caution and consider cross-referencing with "
    "the original books. Feel free to rephrase your question for a more targeted search."
)
```

8.9 Graph Edges

START

```
-> rewrite_and_route
    |-- route=="general" --> general_answer --> build_final_answer --> END
    |-- route=="rag" --> retrieve --> generate --> evaluate_grounding
                                |
                                [is_grounded==True OR retry_count==1]
                                |
                                build_final_answer --> END
                                |
                                [is_grounded==False AND retry_count==0]
                                |
                                expand_query --> retrieve (loop)
```

Router function for evaluate_grounding conditional edge:

```
def route_after_eval(state: RAGState) -> str:
    if state["is_grounded"] or state["retry_count"] >= 1:
        return "build_final_answer"
    return "expand_query"
```

IMPORTANT: The retrieve node must handle both first attempt and retry
by checking state["retry_count"] and using the appropriate query + doc merging.

9. Streamlit App — app.py

Layout (single screen)

+-----+

```

| ■ BookRAG – Your Personal Finance Assistant |
| [Clear chat] [top] |
| ----- |
| |
| [Chat message area – scrollable, full height] |
| |
| ■ User: ... |
| ■ Assistant: ... |
| [■■ Disclaimer if ungrounded – st.warning] |
| [■ Pipeline trace – st.expander, collapsed] |
| |
| ----- |
| [ Type your message... ] [ Send ↵ ] |
+-----+

```

Session State

```

st.session_state.messages # list of {"role", "content", "trace", "disclaimer"}
st.session_state.chat_history # ChatHistory instance
st.session_state.pipeline # compiled LangGraph app (via @st.cache_resource)
st.session_state.retriever # RetrieverClient (via @st.cache_resource)

```

Pipeline Trace Expander

Each assistant message has an optional trace dict rendered inside `st.expander('■ Pipeline trace', expanded=False)`:

```

trace = {
    "route": "rag" | "general",
    "rewritten_query": str,
    "retry_count": int,
    "sources": [{"book_title": str, "rrf_score": float}],
    "is_grounding": bool,
    "grounding_reason": str,
}

# Render:
# - Route badge: ■■ General or ■ Knowledge Base
# - Rewritten query (shown if different from original)
# - Sources: book titles + RRF scores (if RAG path)
# - Grounding: ■ Verified or ■■ Unverified – {grounding_reason}
# - Retry: ■ Query was expanded and retried once (if retry_count == 1)

```

Startup Guard

```

import os
if not os.path.exists("./chroma_db/bm25.pkl"):
    st.error(
        "■■■ Vector database not found. "
        "Please run `python ingest.py` first, then restart the app."
    )
st.stop()

```

Caching

```

@st.cache_resource
def load_pipeline():
    from pipeline import build_graph
    return build_graph() # LLM clients created inside, cached across reruns

```

```
def load_retriever():
    from utils.retrieval import RetrieverClient
    return RetrieverClient()
```

10. Global Assumptions for Antigravity

#	Assumption	Fallback if wrong
1	Ollama endpoint is OpenAI-compatible at {OLLAMA_BASE_URL}	Leave # TODO: update base_url format comment
2	No auth header needed on ngrok URL	Leave # TODO: add Authorization header comment in LLM client init
3	All 11 PDFs have copyable text (confirmed by user — no OCR needed)	OCR every text() per page; skip empty pages silently
4	Single user, single session — no multi-user support needed	Use session_state is sufficient for state isolation
5	BM25 corpus, metadata, and Chroma are built in the same repo	no loop — metadata alignment maps BM25 feed → book metadata explicitly
6	Retry cap is exactly 1 — pipeline never loops more than once	Have to debug RAG retry_count >= 1 always routes to build_final_answer
7	No streaming — responses returned as complete strings	st.spinner() covers wait time; keep it simple for reliability
8	thinking mode disabled — do not use tool-calling/function-calling	Use plain text and JSON output with code-fence stripping for all structured calls
9	Chunk overlap is generous (150 tokens) to prevent ideas from splitting	Try splitting into more chunks, slightly longer ingest — acceptable for quality goal
10	On JSON parse failure: route='rag', rewritten_query=raw_query, validate_output=False	Conservative defaults always prefer retry over skipping

11. File Build Order for Antigravity

Build in this exact order to catch dependency issues early:

```
1. requirements.txt
2. .env.example
3. utils/__init__.py          (empty file)
4. utils/embeddings.py
5. utils/retrieval.py
6. utils/history.py
7. ingest.py
8. pipeline.py
9. app.py
10. README.md
```

setup_and_run.sh — generate this file too

```
#!/bin/bash
pip install -r requirements.txt
echo ""
echo "■ Dependencies installed."
echo ""
echo "Next steps:"
echo " 1. Copy .env.example to .env and set OLLAMA_BASE_URL"
echo " 2. Place your 11 PDF books in ./books/"
echo " 3. Run: python ingest.py"
echo " 4. Run: streamlit run app.py"
```

12. README Content (include in README.md)

Setup

```
git clone <repo>
cd bookrag
pip install -r requirements.txt
cp .env.example .env
# Edit .env – set OLLAMA_BASE_URL to the current ngrok URL
```

Ingest Books

```
# Place your 11 PDF files in ./books/ then:
python ingest.py
```

Run

```
streamlit run app.py
```

Architecture

```
User Input
|
v
[Streamlit UI]
|
v
[LangGraph Pipeline]
|
+---> [0.8b] Rewrite query + route decision
|
+--- GENERAL PATH ---> [2b] Direct LLM answer
|
+--- RAG PATH
|
v
[Chroma dense retrieval, top-15]
+
[BM25 sparse retrieval, top-15]
|
v
[RRF Fusion – top-8 docs]
|
v
[4b] Generate answer (strict: excerpts only)
|
v
[4b] Evaluate grounding (strict checker)
|
+-----+-----+
|         |         |
Grounded   Not grounded (attempt 1 only)
|         |         |
v         v         v
Final answer [0.8b] Expand query
              + merge old docs
              |
              v
[Retry retrieval – cumulative docs]
|
```

v
[4b] Generate again
|
v
[4b] Evaluate again
|
Grounded or not --> Final answer
(with disclaimer if still failing after retry)

End of spec. Antigravity: build all files now, make all decisions, leave no TODOs except the optional auth one noted in
Section 10. The user will run python ingest.py then streamlit run app.py and it must work.